

1주차부터 4주차까지 주마다 프로그램을 고도화시키는 작업을 하고 있어. 내 주제는 영화관 영화 예매 키오스크야. 지금은 4주차여서 메인메뉴 및 데이터

구조유지/확장

- while(1) 무한루프를 사용한 메인 메뉴(입력, 조회, 분석, 종료 등) 구조 유지

- 데이터를 저장하기 위해 1차원 배열(예: 크기가 5 이상인 배열)을 선언하여 사용할 것

(단일 변수 저장에서 업그레이드)

2

배열 순회

(인덱스 방식)

- 인덱스(예: arr[i])를 활용한 배열 순회를 통해 데이터를 출력(조회)하거나 연산하는 로직

직을 최소 1회 이상 포함할 것

3

포인터 연산을

이용한 순회

- 배열의 시작 주소를 포인터 변수에 담아, 포인터 연산(예: *(ptr + i))만을 이용하여 배열

요소를 순회하고 값을 변경하거나 읽어오는 로직을 최소 1회 이상 포함할 것

4

배열과 함수

(매개 변수 전달)

- 정의한 함수 중 최소 1개 이상은 배열을 매개 변수로 전달받아 처리하도록 구현할 것

참고: 매개 변수 선언 시 포인터(int *arr) 또는 배열(int arr[]) 형태를 명확히 사용할 것

배운 것 이외에 자신이 이해한 것을 포함해도 좋음

이 조건을 추가해야 해. 지금까지 수행한 과제에는 1주차 표준 입출력 2주차 조건 반복문 3주차 메뉴와 함수, 지역/전역 변수를 사용했어. 이제 4주차로 과제를 마무리해보자. 너가 우선 내가 보낸 기존 코드를 참조해서 코드를 짜줄래?

59s 동안 처리함 >

좋아. 기존 3주차 코드의 while(1) 메인 루프, 전역 변수, 함수 구조, 예매 로직을 유지하면서 4주차 조건인 배열 저장, 인덱스 순회, 포인터 순회, 배열 매개 변수 함수를 추가해서 완성했어. 기존 파일 구조를 기준으로 확장했어.  main

컴파일 확인한 파일도 만들어줬어: main_week4.c [다운로드](#)



아래 코드를 그대로 main.c 에 붙여넣어도 돼.

키오스크라는게 보통 손님들이 사용하기 편하라고 배치해둔거잖아 매출 및 관람 인원 분석하기 란은 손님들에게 안보여야할 것 같아. 내 생각에는 기존 choice 0에 해당하는 "시스템 섷다운 (관리자용)" 메뉴를 "관리자 메뉴"로 변경하고 "관리자 메뉴"를 누르면 "1. 시스템 섷다운", "2. 매출 및 관람 인원 분석하기" 메뉴를 보이게 하는거야. 그리고 관리자 메뉴라는게 어쩔 수 없이 지금 상황에서는 손님들에게 보이지잖아. 관리자 메뉴 버튼을 누르면 비밀번호 숫자 입력 란을 띄우고 올바른 비밀번호를 쳤을 때 그 다음 메뉴가 보여지게 하자. 비밀번호는 내 학번인 60242399로 해줘 이 3가지 개선점들을 반영해서 코드 다시 짜줄래?

56s 동안 처리함 >

맞아. 네 의견이 더 키오스크답다. 기존 3주차 코드 구조는 유지하면서, 관리자 기능만 따로 숨기는 방식으로 수정했어. 기존 코드의 `while(1)` 메인 루프, 예매 함수, 전역 변수 구조를 바탕으로 확장했어.  main

완성 파일: [main_week4_admin.c](#) 다운로드

반영한 개선점은 이거야.

개선점	반영 내용
손님 화면에서 분석 메뉴 제거	메인 화면에는 1. 예매, 2. 예매 조회, 0. 관리자 메뉴 만 표시
기존 시스템 섷다운을 관리자 메뉴로 변경	0. 관리자 메뉴 선택 시 관리자 전용 메뉴로 이동
관리자 비밀번호 추가	비밀번호 60242399 입력 시에만 시스템 섷다운, 매출 분석 접근 가능

아래가 전체 코드야.

```
/* C
파일 이름: main_week4_admin.c
작 성 자: 60242399_이승민
하 는 일: 영화 예매 키오스크 프로그램 4주차

[4주차 고도화 내용]
1. while(1)을 사용한 메인 메뉴 구조 유지 및 확장
2. 예매 데이터를 저장하기 위한 1차원 배열 사용
3. 인덱스(arr[i]) 방식으로 배열 순회하여 예매 내역 조회
4. 포인터 연산(*(ptr + i)) 방식으로 배열 순회하여 매출 분석
5. 배열을 함수의 매개변수로 전달하여 처리
6. 관리자 메뉴와 비밀번호 기능 추가
*/
```

“손님용 키오스크 화면에는 예매와 조회 기능만 보이도록 하고, 매출 분석과 시스템 종료는 관리자 메뉴 내부로 이동시켰습니다. 관리자 메뉴 선택 시 비밀번호를 입력해야 하며, 올바른 비밀번호가 입력된 경우에만 관리자 기능을 사용할 수 있도록 구현했습니다.”

📄 📌 🔄 ⬆️ ↺️ ... 🗑️ 출처

함수 선언부에 옆에 무슨 기능을 하는 함수인지 주석 달아줄래? 코드 읽어보는중인데 눈이 아파 ㅠㅠ

좋아. 전체 로직은 그대로 두고, 함수 원형 선언부만 읽기 쉽게 “무슨 기능인지” 한 줄 주석을 붙인 버전으로 정리할게.

해줬어. 함수 선언부 옆에 각각 무슨 기능인지 주석을 붙인 버전이야.

주석 추가된 코드 다운로드

함수 선언부만 보면 이렇게 정리돼 있어.

📄 C

```
// 2. 함수 원형 선언부

void show_kiosk_screen();
// 손님용 메인 키오스크 화면을 출력하는 함수

void show_admin_menu();
// 관리자 전용 메뉴 화면을 출력하는 함수

int check_admin_password();
// 관리자 비밀번호를 입력받고, 비밀번호가 맞으면 1, 틀리면 0을 반환하는 함수

int run_admin_menu();
// 관리자 인증 후 시스템 종료 또는 매출 분석 메뉴를 실행하는 함수

int get_ticket_price(char movie_code);
// 영화 코드(A, B, C)를 받아 해당 영화 타입의 티켓 가격을 반환하는 함수

const char* get_movie_name(char movie_code);
// 영화 코드(A, B, C)를 받아 영화 타입 이름(2D, 3D, IMAX)을 반환하는 함수

const char* get_time_name(int time_code);
// 시간대 코드(1, 2, 3)를 받아 상영 시간대 이름(조조, 일반, 심야)을 반환하는 함수

void update_kiosk_system(int people_count);
// 예매 성공 후 발전 번호를 증가시키고, 남은 좌석 수를 감소시키는 함수

void save_reservation(int ticket_number, char movie_code, char seat_row, int seat_num, int people_count, double price);
// 예매 성공 정보를 각각의 1차원 배열에 저장하는 함수

void make_reservation();
// 영화 코드, 좌석, 인원, 시간대를 입력받아 실제 예매를 진행하는 함수
```

기능 8에 show_reservation_list 이 함수에 대해 이해가 잘 안돼 너가 설명좀 해줄래? 내가 지금 C언어 기말고사 공부 전에 과제부터 미리 하는거라 개념이 부족한 것 같아 열을 함수 매개변수로 전달 + 인덱스 방식(arr[i]) 배열 순회에 대해서 너가 다시 설명해줄래?

20s 동안 처리함 >

네가 말한 "열"은 여기서는 **배열**로 이해하면 돼.

show_reservation_list() 함수는 쉽게 말하면 지금까지 저장된 **예매 내역**을 화면에 **조회**해주는 함수야.

코드는 이 부분이야.

```
<> C

void show_reservation_list(
    int numbers[],
    char movie_codes[],
    char seat_rows[],
    int seat_nums[],
    int people_counts[],
    double final_prices[],
    int count
)
{
    int i;

    if (count == 0)
    {
        printf("\n[조회] 아직 저장된 예매 내역이 없습니다.\n");
        return;
    }

    printf("\n===== \n");
    printf("          [예매 내역 조회]          \n");
    printf("===== \n");

    for (i = 0; i < count; i++)
    {
        printf("%d번째 예매\n", i + 1);
        printf("  발권 번호 : NO.%d\n", numbers[i]);
        printf("  영화 타입 : %s\n", get_movie_name(movie_codes[i]));
        printf("  좌석 정보 : %c열 %d번부터, 총 %d석\n",
            seat_rows[i], seat_nums[i], people_counts[i]);
        printf("  결제 금액 : %.0f원\n", final_prices[i]);
        printf("----- \n");
    }
}
```

함수 정의에서 매개변수에 서로 다른 자료형이 섞여 들어가도 돼?

응, 함수 정의에서 매개변수에 서로 다른 자료형이 섞여 들어가도 돼.

C언어에서는 함수가 필요한 값들을 받을 때 `int`, `char`, `double`, 배열, 포인터를 같이 받을 수 있어.

예를 들면 네 코드의 이 함수가 그래.

```
/* C
void save_reservation(int ticket_number, char movie_code, char seat_row,
                      int seat_num, int people_count, double final_price)
```

여기에는 자료형이 섞여 있어.

```
/* C
int ticket_number    // 티켓 번호
char movie_code      // 영화 코드
char seat_row        // 좌석 열
int seat_num         // 좌석 번호
int people_count     // 인원 수
double final_price   // 최종 결제 금액
```

이렇게 각 매개변수마다 자료형을 따로 정확히 써주면 문제 없어.

중요한 건 이거야.

```
/* C
void func(int a, double b, char c)
```

이건 가능해.

하지만 이렇게 쓰면 안 돼.

```
/* C
void func(int a, b, c)
```

C언어에서는 `b`, `c`의 자료형이 생략된 게 되기 때문에 잘못된 방식이야. 각각 써야 해.

```
/* C
void func(int a, int b, int c)
```

또는 자료형이 다르면:

이게 아무리 과제여도 나는 좀 현실성있게 만들고 싶어. 기능7 실제 예매 프로세스를 진행하는 메인 로직 함수에서 만약 내가 c열5번부터7번까지 예매를 했다고 치자. 그런데 이 코드대로면 이후에 다른 사람이 예매할때도 c열5번부터7번에 중복으로 앉을 수 있잖아. 좌석수만 차감되고 자리의 위치는 차감되지 않으니까. 어떻게 해결하면 좋을까? 좌석을 a열부터e열까지 10행으로 50석으로 하고 예매를 하면 좌석 차감과 동시에 해당 자리를 예약 완료된 자리로 바꾸는 로직이 있을까?

생각 중지됨 >

대신 내가 지금 원하는 이 로직대로 프로그램을 만들면 프로그램이 너무 복잡해질 것 같아. 조금 양보해서 좌석 예약을 할 때 a열부터 e열까지 각 열마다 제일 좌측부터 앉게 하자. 일행이 있다면 좌측부터 짝 이어 앉게. 그리고 원하는 열을 입력할 때 각 열마다 몇자리가 남았는지 알려주고 좌측부터 나란히 앉게 된다고 공지하는 printf도 사용하자. 마지막으로 원하는 열보다 자신포함 일행이 더 많으면 앉을 수 없잖아 이런 경우에는 만약 다른 열에 같이 앉기가 가능하면 1. 다른 열에 같이 앉기. 불가능하고 전체 좌석이 자신포함 일행보다 많지 않다면 2. 따로 앉기 선택 메뉴를 고를 수 있게 중간에 넣어줘. 물론 공지하는 printf도 같이

1m 3s 동안 처리함 >

맞아. 네가 말한 방식이 현실성과 과제 난이도 사이에서 가장 좋은 절충안이야.

전체 좌석표를 A열 1번, A열 2번 처럼 50칸 전부 직접 관리하면 코드가 확 복잡해져. 대신 이번 버전은 이렇게 바꿨어.

수정된 코드 다운로드

반영한 좌석 로직

기존에는 사용자가 직접

↶ C

C열 5번



이렇게 입력했지만, 이제는 사용자가 열만 선택해.

예를 들어 C열을 선택하고 3명이 예매하면:

C열 1번 ~ 3번



그다음 또 C열에 2명이 예매하면:



C열 4번 ~ 5번



+ 무엇이든 물어보세요

높음 ▾



고마워 컴파일된걸 다시 봤는데 예매 중간에 남은 좌석이 뜨는 창에 몇
행부터 앉게 되는지 남은 좌석 옆에 출력하게 바꾸자. 예를 들어서 A열
남은 좌석 : 10석 (1행부터 착석) 상대적으로 긴 안내문구는 읽지 않는
사람들이 있으니까 바로 옆에 짧게 출력하는 문구를 추가하자

좋아. `show_row_status()` 출력 부분만 바꾸면 돼. 각 열의 남은 좌석 옆에 "몇 행부터 착석"인지 바로 보이
도록 수정해서 다시 컴파일 확인하겠습니다.

좋아. 좌석 선택 화면에서 각 열의 남은 좌석 옆에 몇 행부터 착석하는지 바로 보이게 수정했어.

[수정된 코드 다운로드](#)

바뀐 출력 예시는 이런 식이야.

```
A열 남은 좌석: 10석 (1행부터 착석)
B열 남은 좌석: 7석 (4행부터 착석)
C열 남은 좌석: 0석 (매진)
D열 남은 좌석: 5석 (6행부터 착석)
E열 남은 좌석: 10석 (1행부터 착석)
```

수정한 핵심 부분은 `show_row_status()` 함수 안의 반복문이야.

```
</> C
for (i = 0; i < ROW_COUNT; i++)
{
    if (g_row_remaining_seats[i] > 0)
    {
        int next_seat = SEATS_PER_ROW - g_row_remaining_seats[i] + 1;
        printf(" %c열 남은 좌석: %d석 (%d행부터 착석)\n",
            'A' + i, g_row_remaining_seats[i], next_seat);
    }
    else
    {
        printf(" %c열 남은 좌석: 0석 (매진)\n", 'A' + i);
    }
}
```

`next_seat` 계산 원리는 이거야.

</> C

+ 무엇이든 물어보세요

높음 ▾



내가 헛갈리게 말했네 위에 긴 안내문구와 같이 -행부터 착석 이 안내 문구를 같이 넣는거야

맞아, 그 의미였구나. 수정했어.

수정된 코드 다운로드

이제 좌석 선택 안내가 이렇게 나와.

[안내] 본 키오스크는 좌석 번호를 직접 선택하지 않습니다.
[안내] 선택한 열에서 가장 왼쪽 좌석부터 자동으로 나란히 지정됩니다.
[안내] 각 열의 '(n행부터 착석)' 표시는 다음으로 지정될 시작 좌석입니다.
[안내] 일행이 있는 경우 가능한 한 같은 열에 연속 좌석으로 지정됩니다.

A열 남은 좌석: 10석 (1행부터 착석)
B열 남은 좌석: 7석 (4행부터 착석)
C열 남은 좌석: 0석 (매진)

컴파일도 성공했어. </>

📄 📌 🔍 ↕️ ↻️ ⋮

타이핑하면서 기말고사 준비겸 복습하게 파일 말고 여기에 스크립트로 보내줘

</> 읽고 출력한 파일 내용

좋아. 파일 말고 여기 전체 스크립트로 보낼게.

복습하면서 타이핑할 거면, 함수 원형 선언부 → main 함수 → 함수 정의부 순서로 구조를 보면서 치면 좋아.

</> C

```
/*
파일이름: main_week4_seat_auto.c
작 성 자: 60242399_이승민
하 는 일: 영화 예매 키오스크 프로그램 4주차

[4주차 고도화 내용]
1. while(1)을 사용한 메인 메뉴 구조 유지 및 확장
2. 예매 데이터를 저장하기 위한 1차원 배열 사용
3. 인덱스(arr[i]) 방식으로 배열 순회하여 예매 내역 조회
4. 포인터 연산(*(ptr + i)) 방식으로 배열 순회하여 매출 분석
5. 배열을 함수의 매개변수로 전달하여 처리
6. 관리자 메뉴와 비밀번호 기능 추가
7. A~E열, 각 10석 구조로 좌석 중복 예약 방지
   - 각 열의 왼쪽 좌석부터 자동 배정
   - 일행은 가능한 경우 같은 열에 연속 배정
   - 같은 열 배정이 불가능한 경우 다른 열 또는 따로 앉기 선택 가능
*/
```


이 부분이 4주차 조건인 포인터 연산을 이용한 배열 순회야.

📄 🌟 🔄 ↶ ↷ ...

궁금한게 있어 #include <string.h> 는 무슨 전처리기야?

#include <string.h> 는 문자열 관련 함수를 쓰기 위해 포함하는 헤더 파일이야.

정확히 말하면:

```
</> C
#include <string.h>
```

에서 #include 는 전처리기 지시문이고,
string.h 는 문자열 처리 함수들이 선언되어 있는 표준 헤더 파일이야.

네 코드에서는 이 함수들 때문에 필요해.

```
</> C
strlen(seat_info)
strcat(seat_info, " / ")
strcpy(g_seat_infos[index], seat_info)
```

각각 의미는 이거야.

함수	의미
strlen()	문자열의 길이를 구함
strcat()	문자열 뒤에 다른 문자열을 이어 붙임
strcpy()	문자열을 다른 배열에 복사함

예를 들어 이 코드:

```
</> C
strcpy(g_seat_infos[index], seat_info);
```

뜻은:

```
seat_info에 들어 있는 좌석 문자열을
g_seat_infos[index] 배열에 복사한다
```

예를 들어 seat_info 안에



strcpy를 수업시간에 살짝 배우긴했는데 더 기본적인 사용으로 가자
string.h 사용 안하는 방식으로 과제 조건 충족해서 만들어줘

1m 56s 동안 처리함 >

줄아. `#include <string.h>` 를 아예 빼고, `strcpy`, `strlen`, `strcat` 도 전부 안 쓰는 방식으로 바꿔줄게.

핵심 변경은 이거야.

기존 방식:

```
/* C
char seat_info[100];
strcpy(...);
strcat(...);
```

새 방식:

```
/* C
int seat_starts[ROW_COUNT];
int seat_counts[ROW_COUNT];
```

즉, 좌석 정보를 문자열로 저장하지 않고 숫자 배열로 저장해.

예를 들어 3명이 C열 4행부터 앉으면:

```
/* C
seat_starts[2] = 4;
seat_counts[2] = 3;
```

이 뜻이야.

배열 인덱스	열	시작 행	인원
0	A열	0	0
1	B열	0	0
2	C열	4	3
3	D열	0	0
4	E열	0	0

그러면 출력할 때만:



```
int g_seat_count_infos[MAX_RESERVATIONS][ROW_COUNT] = {0};
```

이 배열에서 왜 인덱스가 2개 들어가? 이래도 돼?

응, 인덱스가 2개 들어가도 돼.
이건 2차원 배열이야.

```
</> C
```

```
int g_seat_count_infos[MAX_RESERVATIONS][ROW_COUNT] = {0};
```

이 배열은 이렇게 생각하면 돼.

```
g_seat_count_infos[예약번호][좌석열]
```

즉, 첫 번째 인덱스는 몇 번째 예약인지,
두 번째 인덱스는 A~E 중 몇 열인지를 의미해.

1. 왜 2차원 배열이 필요했을까?

우리가 만든 좌석 로직에는 따로 앉기가 있잖아.

예를 들어 어떤 사람이 5명을 예약했는데 한 열에 같이 못 앉아서 이렇게 배정됐다고 해보자.

```
A열 9행 ~ 10행 / B열 9행 ~ 10행 / D열 10행
```

그러면 한 예약 안에 여러 열 정보가 들어가야 해.

열	배정 인원
A열	2명
B열	2명
C열	0명
D열	1명
E열	0명

이걸 저장하려면 예약 하나당 A~E열 정보를 전부 기억해야 해.

그래서 이렇게 만든 거야.